



Politechnika Wrocławska

Platformy programistyczne

Portfolio Tracker

Anna Poberezhna 280096

Dawid Pajor 280067

Grupa nr 1, Wtorek TP 17:05

1 Wprowadzenie

Celem projektu było stworzenie webowej aplikacji do zarządzania portfelem inwestycyjnym, umożliwiającej użytkownikowi wygodne dodawanie, edytowanie i usuwanie aktywów oraz analizę podstawowych danych dotyczących portfela. Aplikacja została wykonana w technologii ASP.NET Core MVC i oparta o architekturę warstwową.

Użytkownik po zalogowaniu ma dostęp do panelu, w którym może zarządzać swoimi aktywami, przeglądać zestawienia oraz korzystać z funkcji raportowania.

2 Zakres funkcjonalny

Aplikacja umożliwia realizację następujących funkcjonalności:

- rejestrację i logowanie użytkowników,
- zarządzanie aktywami inwestycyjnymi (dodawanie, edycja oraz usuwanie aktywów),
- przeglądanie dashboardu zawierającego podsumowanie portfela inwestycyjnego,
- generowanie zestawień oraz raportów dotyczących zgromadzonych aktywów,
- pobieranie danych pomocniczych, w szczególności aktualnych cen kryptowalut,
- zarządzanie użytkownikami oraz rolami w systemie,
- uwierzytelnianie i autoryzację użytkowników w celu zapewnienia bezpieczeństwa danych.

3 Zastosowane technologie i narzędzia

Do realizacji projektu wybrano nowoczesny stos technologiczny oparty na ekosystemie .NET. Dobór narzędzi podyktowany był wymogami dotyczącymi wydajności, bezpieczeństwa oraz czytelnego podziału odpowiedzialności poszczególnych komponentów systemu.

W ramach implementacji wykorzystano następujące technologie:

- **ASP.NET Core MVC** – główny framework aplikacyjny. Stanowi fundament całego systemu, zapewniając ścisły podział odpowiedzialności poprzez wzorzec architektoniczny Model-View-Controller. Ułatwia to organizację kodu, zarządzanie routingu oraz implementację logiki biznesowej.
- **Entity Framework Core** – zaawansowany system mapowania obiektowo-relacyjnego (ORM). Odpowiada za komunikację z bazą danych z wykorzystaniem podejścia *Code-First*. Pozwoliło to na zdefiniowanie struktury danych bezpośrednio w kodzie C# i wyeliminowanie konieczności ręcznego pisania zapytań SQL.
- **SQLite** – lekki, bezserwerowy silnik relacyjnej bazy danych. Został zaimplementowany ze względu na łatwość wdrożenia (ang. *zero-configuration*) w środowisku deweloperskim i testowym, w którym cała struktura bazy przechowywana jest w pojedynczym pliku binarnym.
- **ASP.NET Core Identity** – zintegrowany moduł bezpieczeństwa. W aplikacji odpowiada za kompleksową obsługę kont użytkowników, bezpieczne przechowywanie haseł (dzięki algorytmom haszującym) oraz zarządzanie uwierzytelnianiem i autoryzacją w oparciu o role.
- **Bootstrap oraz jQuery** – technologie warstwy frontendowej (prezentacji). Framework *Bootstrap* wykorzystano do budowy responsywnego interfejsu użytkownika (RWD) i ustandaryzowanych komponentów wizualnych. Z kolei biblioteka *jQuery* posłużyła do obsługi zdarzeń po stronie przeglądarki oraz asynchronicznej manipulacji drzewem DOM.

4 Architektura i struktura aplikacji

4.1 Paradygmat i wzorzec architektoniczny

Aplikacja została zaprojektowana w oparciu o wzorzec **MVC (Model-View-Controller)**, który zapewnia wyraźną separację logiki biznesowej (Model), routingu i logiki prezentacji (Controller) oraz samej warstwy interfejsu użytkownika (View).

Zastosowano przejrzysty wzorzec warstwowy, dzieląc system na następujące obszary:

- **Warstwa Prezentacji:** widoki oparte na silniku Razor (Razor Views).
- **Warstwa Aplikacji:** kontrolery oraz dedykowane serwisy (Services).

- **Warstwa Danych:** zaimplementowana przy użyciu Entity Framework Core oraz klasy `DbContext`.
- **Warstwa Infrastruktury:** konfiguracja, mechanizmy tożsamości (ASP.NET Identity), pamięć podręczna (cache) oraz klienci HTTP.

4.2 Główne komponenty i struktura plików

Struktura projektu odzwierciedla podział na warstwy, a kluczowe punkty systemu znajdują się w następujących plikach i katalogach:

- **Punkt startowy:** Plik `Program.cs` odpowiada za inicjalizację, konfigurację potoku żądań (middleware) oraz rejestrację usług.
- **Kontrolery:** Obsługują żądania HTTP i sterują przepływem danych. Przykłady to `AssetsController.cs` oraz `HomeController.cs`.
- **Modele domenowe:** Reprezentują encje bazodanowe oraz modele widoków (np. `Asset.cs`, `DashboardViewModel.cs`).
- **Warstwa danych:** Klasa `ApplicationDbContext.cs` zarządza encjami i migracjami bazy danych. Wykorzystano silnik SQLite, którego konfiguracja znajduje się w pliku `appsettings.json`.
- **Usługi aplikacyjne (Services):** Hermetyzują logikę niezależną od protokołu HTTP. Są one wstrzykiwane przez mechanizm Dependency Injection (np. `CryptoService.cs`, `ReportService.cs`).
- **Widoki i zasoby UI:** Katalog `Views` zawiera strony Razor, natomiast `wwwroot` przechowuje pliki statyczne (CSS, JS).
- **Konfiguracja:** Pliki `appsettings.json` oraz `appsettings.Development.json` definiują parametry środowiskowe, takie jak ciągi połączeń (connection strings).

4.3 Zależności, konfiguracja i warstwa danych

Projekt silnie wykorzystuje mechanizm wstrzykiwania zależności (ang. *Dependency Injection*). Główne parametry pracy utrzymywane są w pliku konfiguracyjnym, natomiast wszystkie kontrolery, usługi zewnętrzne oraz moduły tożsamości rejestrowane są centralnie podczas startu aplikacji.

Poniżej przedstawiono zawartość pliku `appsettings.json`, w którym zdefiniowano ciąg połączenia do wbudowanej bazy SQLite (`DefaultConnection`) oraz bazową konfigurację poziomów logowania zdarzeń:

```

1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Data Source=portfolio.db"
4   },
5   "Logging": {
6     "LogLevel": {
7       "Default": "Information",
8       "Microsoft.AspNetCore": "Warning"
9     }
10  },
11  "AllowedHosts": "*"
12 }

```

Listing 1: Konfiguracja środowiskowa – plik appsettings.json

Kluczowym elementem sterującym pracą całej aplikacji jest plik `Program.cs`. Pełni on rolę punktu wejściowego, w którym następuje rejestracja usług w kontenerze IoC (w tym serwisów domenowych takich jak `CryptoService` oraz `ReportService`), konfiguracja bazy danych, ustawienie polityki walidacji hasła dla modułu ASP.NET Core Identity oraz zdefiniowanie początkowych ról systemowych (mechanizm Data Seeding).

Poniższy listing prezentuje kompletną konfigurację potoku żądań HTTP (Middleware) oraz rejestrację zależności:

```

1 using Microsoft.AspNetCore.Identity;
2 using Microsoft.EntityFrameworkCore;
3 using PortfolioTracker.Data;
4 using PortfolioTracker.Services;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 // Add services to the container.
9 builder.Services.AddControllersWithViews();
10 builder.Services.AddMemoryCache();
11 builder.Services.AddHttpClient<CryptoService>();
12 builder.Services.AddScoped<PortfolioTracker.Services.
    CryptoService>();
13 builder.Services.AddScoped<PortfolioTracker.Services.
    ReportService>();
14
15 builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
16     options.UseSqlite(builder.Configuration.
        GetConnectionString("DefaultConnection")));
17
18 // Add Identity

```

```

19 builder.Services.AddIdentity<IdentityUser, IdentityRole>(
    options =>
20 {
21     options.Password.RequireDigit = false;
22     options.Password.RequiredLength = 6;
23     options.Password.RequireNonAlphanumeric = false;
24     options.Password.RequireUppercase = false;
25     options.Password.RequireLowercase = false;
26 })
27 .AddEntityFrameworkStores<ApplicationDbContext>()
28 .AddDefaultTokenProviders();
29
30 var app = builder.Build();
31
32 // Configure the HTTP request pipeline.
33 if (!app.Environment.IsDevelopment())
34 {
35     app.UseExceptionHandler("/Home/Error");
36     // The default HSTS value is 30 days. You may want to
    // change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
37     app.UseHsts();
38 }
39
40 app.UseHttpsRedirection();
41 app.UseRouting();
42
43 app.UseAuthentication();
44 app.UseAuthorization();
45
46 app.MapStaticAssets();
47
48 app.MapControllerRoute(
49     name: "default",
50     pattern: "{controller=Home}/{action=Index}/{id?}")
51     .WithStaticAssets();
52
53 // Seed roles
54 using (var scope = app.Services.CreateScope())
55 {
56     var roleManager = scope.ServiceProvider.
    GetRequiredService<RoleManager<IdentityRole>>();
57     var roles = new[] { "Admin", "Standard User" };
58
59     foreach (var role in roles)
60     {
61         if (!await roleManager.RoleExistsAsync(role))
62         {
63             await roleManager.CreateAsync(new IdentityRole

```

```

        (role));
64     }
65     }
66 }
67
68 app.Run();

```

Listing 2: Punkt startowy i konfiguracja potoku – plik Program.cs

Bezpośrednim przedłużeniem konfiguracji startowej jest warstwa dostępu do danych (Data Access Layer). Jej głównym węzłem komunikacyjnym jest klasa **ApplicationDbContext**. Dziedziczenie po wbudowanej klasie **IdentityDbContext** zapewnia natywne wsparcie dla tabel systemowych modułu tożsamości (użytkownicy, role, tokeny).

Ponadto klasa ta definiuje tabele domenowe (**DbSet**) oraz wykorzystuje mechanizm Fluent API do precyzyjnej konfiguracji relacji kluczy obcych i reguł usuwania powiązanych rekordów (ang. *Cascade Delete*), co zaprezentowano na poniższym listingu:

```

1  using Microsoft.AspNetCore.Identity;
2  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
3  using Microsoft.EntityFrameworkCore;
4  using PortfolioTracker.Models;
5
6  namespace PortfolioTracker.Data
7  {
8      public class ApplicationDbContext : IdentityDbContext
9      {
10         // Constructor
11         public ApplicationDbContext(DbContextOptions<
12             ApplicationDbContext> options)
13             : base(options)
14         {
15
16             public DbSet<Asset> Assets { get; set; }
17             public DbSet<UserActivity> UserActivities { get;
18                 set; }
19
20             protected override void OnModelCreating(
21                 modelBuilder builder)
22             {
23                 base.OnModelCreating(builder);
24
25                 // Configure Asset to User relationship
26                 builder.Entity<Asset>()

```

```

25         .HasOne<IdentityUser>()
26         .WithMany()
27         .HasForeignKey(a => a.UserId)
28         .OnDelete(DeleteBehavior.Cascade);
29
30         // Configure UserActivity to User relationship
31         builder.Entity<UserActivity>()
32             .HasOne(ua => ua.User)
33             .WithOne()
34             .HasForeignKey<UserActivity>(ua => ua.
35                 UserId)
36             .OnDelete(DeleteBehavior.Cascade);
37     }
38 }

```

Listing 3: Definicja kontekstu bazy danych i relacji – plik ApplicationDbContext.cs

4.4 Bezpieczeństwo i autoryzacja

Za bezpieczeństwo aplikacji odpowiada zintegrowany moduł ASP.NET Identity, który obsługuje bezpieczne przechowywanie haseł (hashowanie) oraz zarządzanie rolami użytkowników. Dostęp do wrażliwych zasobów i endpointów modyfikujących dane jest chroniony deklaratywnie za pomocą atrybutu `[Authorize]`. Wdrożono również dobre praktyki, takie jak walidacja danych po stronie serwera oraz ściśle filtrowanie danych przekazywanych do warstwy widoku.

4.5 Decyzje projektowe i rozszerzalność

Wybór stosu technologicznego (ASP.NET MVC + EF Core) został podyktowany potrzebą szybkiego i niezawodnego wdrożenia aplikacji typu CRUD z rozbudowanym modułem raportowania. Wydzielenie logiki do osobnych serwisów zapewnia wysoką testowalność (np. poprzez mockowanie zewnętrznych API). Użycie bazy SQLite upraszcza deployment, a zastosowanie Entity Framework Core gwarantuje, że ewentualna zmiana silnika bazy danych (np. na PostgreSQL) w przyszłości będzie wymagała jedynie minimalnych zmian konfiguracyjnych.

5 Interfejs aplikacji

5.1 Technologie i układ bazowy

Interfejs użytkownika został zrealizowany w oparciu o silnik generowania widoków Razor (pliki `.cshtml`) z wykorzystaniem frameworka Bootstrap w celu zapewnienia pełnej responsywności (Responsive Web Design) oraz biblioteki jQuery do obsługi asynchronicznych zdarzeń po stronie klienta.

Centralnym punktem struktury nawigacyjnej jest plik układu `_Layout.cshtml`. Definiuje on główny pasek nawigacyjny (Navbar), zawiera odnośniki do kluczowych modułów systemu (Home, My Assets), sekcję zarządzania sesją (logowanie/rejestracja) oraz globalny przełącznik motywu graficznego (Dark Mode/Light Mode).

5.2 Główne widoki systemu

Aby kompleksowo zaprezentować możliwości platformy, poniżej opisano kluczowe ekrany aplikacji wraz z odniesieniami do odpowiadających im plików źródłowych. W miejscach oznaczonych jako zrzuty ekranu umieszczono wizualizacje gotowego interfejsu.

5.2.1 Dashboard (Strona główna)

Plik źródłowy: `Dashboard.cshtml`

Dashboard stanowi centrum analityczne inwestora. Na ekranie zaimplementowano cztery karty metryczne podsumowujące kluczowe wskaźniki: Total Value, Total Gain/Loss, Total Assets oraz Today's Change. Kondycja portfela prezentowana jest wizualnie za pomocą wykresu liniowego (obrazującego wartość w czasie) oraz wykresu kołowego (wskazującego alokację i dywersyfikację aktywów). Poniżej znajduje się dedykowana sekcja "Quick Actions" oraz miniaturowa tabela z szybko zmieniającymi się cenami walorów.

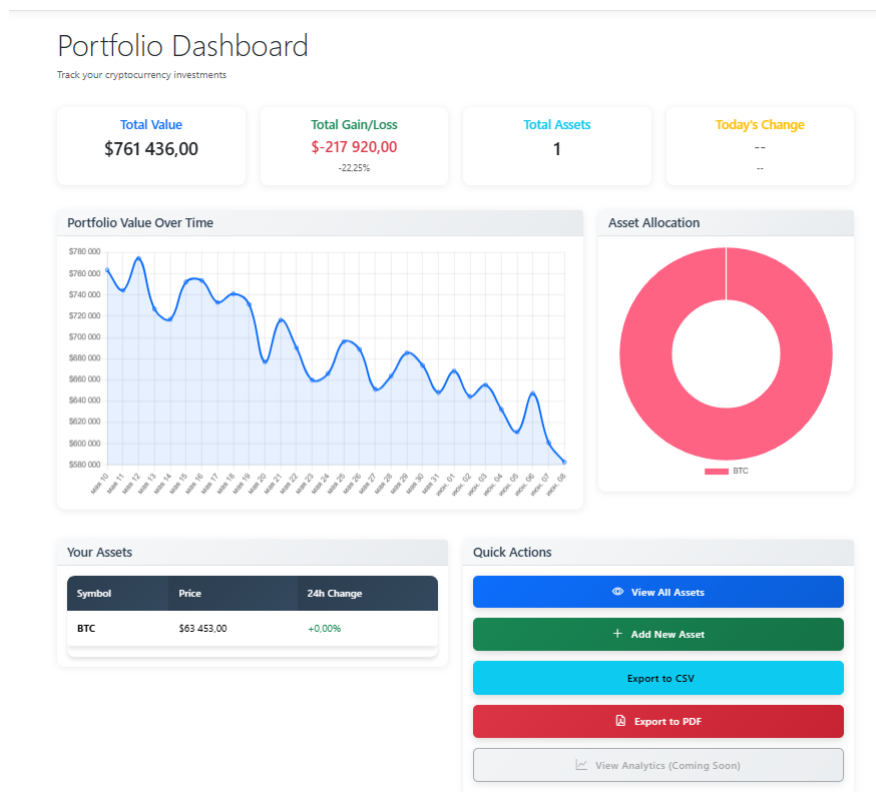
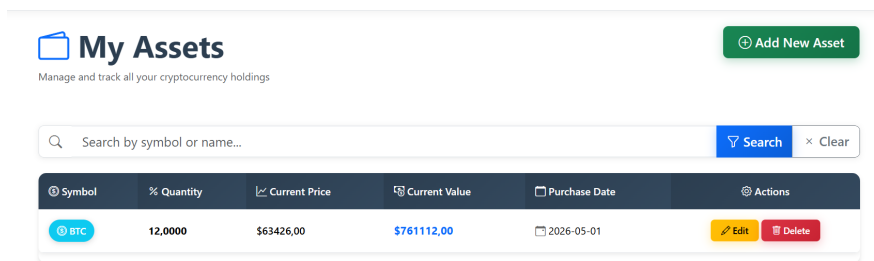


Figure 1: Ekran główny (Dashboard) przedstawiający podsumowanie portfela

5.2.2 Lista aktywów (My Assets)

Plik źródłowy: `Index.cshtml`

Szczegółowy widok tabelaryczny portfela. Tabela zawiera kolumny z danymi zasilanymi na żywo: Symbol, Quantity, Current Price, Current Value oraz Purchase Date. W osobnej kolumnie "Actions" umieszczono przyciski operacyjne (Edit/Delete). Moduł wyposażono w dynamiczną wyszukiwarkę do filtrowania zasobów oraz wyeksponowany przycisk "Add New Asset".



My Assets
Manage and track all your cryptocurrency holdings

[Add New Asset](#)

Search by symbol or name... [Search](#) [Clear](#)

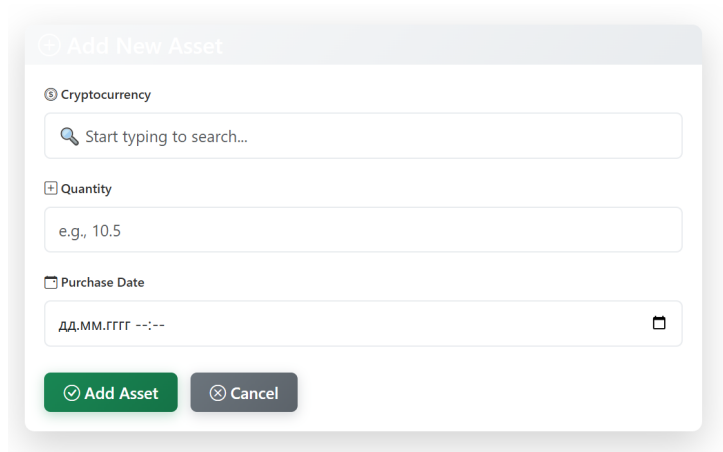
Symbol	% Quantity	Current Price	Current Value	Purchase Date	Actions
BTC	12,0000	\$63426,00	\$761112,00	2026-05-01	Edit Delete

Figure 2: Widok tabelaryczny aktywów (My Assets)

5.2.3 Dodawanie nowego aktywa (Add New Asset)

Plik źródłowy: `Create.cshtml`

Formularz umożliwiający rejestrację nowej transakcji. W celu ułatwienia pracy, pole "Symbol" wyposażono w natywny mechanizm podpowiedzi oparty na elemencie HTML `<datalist>`, zawierający listę najpopularniejszych kryptowalut i akcji. Formularz wykorzystuje natywny selektor daty (`datetime-local`) i jest zabezpieczony przed atakami typu Cross-Site Request Forgery (CSRF) mechanizmem Anti-forgery.



Add New Asset

Cryptocurrency

Start typing to search...

Quantity

e.g., 10.5

Purchase Date

dd.mm.yyyy --:--

[Add Asset](#) [Cancel](#)

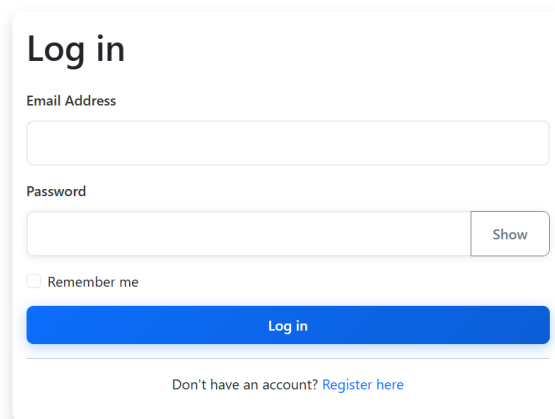
Figure 3: Formularz dodawania nowego aktywa finansowego

5.2.4 Moduł Logowania i Rejestracji

Plik źródłowy: `Login.cshtml`

Czytelny, minimalistyczny formularz autoryzacyjny. Dla maksymalizacji użyteczności zaimplementowano w nim skrypt JavaScript pozwalający na tymczasowe od-

krycie wpisywanego hasła (Show/Hide password) oraz wyraźny odnośnik kierujący niezalogowanych użytkowników do formularza rejestracyjnego.



The image shows a 'Log in' form with a white background and rounded corners. At the top, the title 'Log in' is in a bold, dark font. Below it, there are two input fields: 'Email Address' and 'Password'. The 'Password' field has a 'Show' button to its right. Below the password field is a checkbox labeled 'Remember me'. At the bottom of the form is a large blue button with the text 'Log in'. Below the button, there is a link that says 'Don't have an account? Register here'.

Figure 4: Logowanie

5.3 Personalizacja interfejsu — tryb ciemny (Dark Mode)

Wychodząc naprzeciw współczesnym standardom projektowania interfejsów użytkownika (UX), aplikacja została wyposażona w pełną obsługę motywu ciemnego (*Dark Mode*). Funkcjonalność ta jest dostępna globalnie z poziomu paska nawigacyjnego i pozwala użytkownikowi na natychmiastowe przełączenie stylistyki całej platformy.

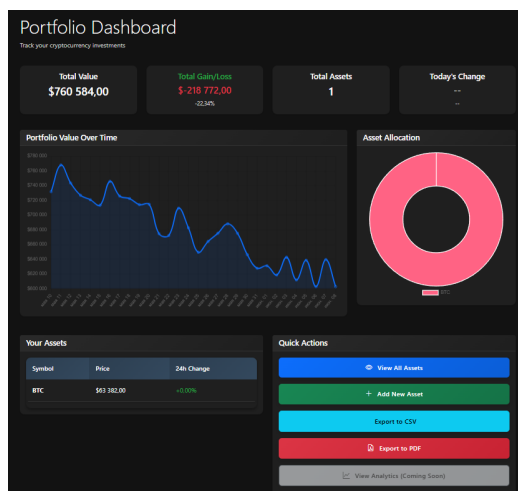


Figure 5: Dark mode - Dashboard